ANTHROPIC CERTIFICATION PREP

Claude Certified Architect

Foundations (CCA-F)

A Complete 1-Month Self-Study Package

Study plan • First-principles notes • Practice questions • Full mock exam • Revision checklist

Target exam	CCA — Foundations (60 questions, 120 min, proctored)
Audience	Newcomers ramping up to architecture-level Claude skills
Schedule	Approx. 1 hour per weekday + a longer hands-on weekend block
Edition	Built May 2026 — verify all facts against the official Exam Guide

Contents

0 Orientation

Read this first. Five minutes here saves you hours later.

What the CCA-Foundations actually is

The Claude Certified Architect — Foundations is Anthropic's first official technical certification, launched in March 2026 through the Anthropic Academy (on the Skilljar platform). It is a proctored, closed-book exam: no Claude, no documentation, and no external tools are allowed during the test. It is designed to verify that you can **design and ship production-grade systems** that use Claude as a core component — think of it as the architecture-level credential for the Claude ecosystem, broadly comparable in spirit to a cloud-provider "Solutions Architect" exam.

Honest expectation-setting (please read)

Officially this is positioned as a **301-level** exam for people *already building* with Claude. It is not a beginner quiz you can skim the night before. A true newcomer absolutely can pass it — but the path is **learn the fundamentals first, then build something real, then test yourself.** This guide is built around exactly that ramp. The single biggest predictor of passing is hands-on experience, so every week includes a build task, not just reading.

Exam facts at a glance

Attribute	Detail
Format	60 multiple-choice / scenario questions
Time	120 minutes (about 2 minutes per question)
Delivery	Online proctored; closed-book, no AI assistance, no docs
Scoring	1,000-point scale (official pass mark not publicly confirmed at time of writing)
Cost	USD $99 — waived for early Claude Partner Network members
Platform	Anthropic Academy (Skilljar)
Style	Systems-design judgement, not syntax recall

The five competency domains

Every question maps to one of five domains. The weightings tell you exactly where to spend your hour. More than 45% of the exam sits in the first two domains, so they get the most study time in the plan that follows.

#	Domain	Weight	What it really tests
1	Agentic Architecture & Orchestration	27%	Designing multi-agent and tool-using systems that recover from failure
2	Claude Code Configuration & Workflows	20%	Configuring Claude Code for real teams and codebases
3	Prompt Engineering & Structured Output	20%	Reliable, parseable output at scale
4	Tool Design & MCP Integration	18%	Safe, well-scoped tools via the Model Context Protocol
5	Context Management & Reliability	15%	Token budgets, RAG, caching, long-run coherence

Note on accuracy

Domain names and weightings reflect Anthropic's public announcement materials as of mid-2026. Exam content and weightings can change. **Always download and follow the official CCA-Foundations Exam Guide from the Anthropic Academy portal** as your source of truth — treat this package as a structured companion, not a substitute.

How to use this guide

- **Follow the calendar in Section 1.** It is sized for roughly one focused hour on each weekday, with a longer hands-on block on the weekend where the real learning happens.
- **Read the matching notes in Section 2** on the day they are scheduled. They are written from first principles, so you do not need prior Claude experience to start.
- **Do the practice questions in Section 3** at the end of each domain week. Mark yourself honestly and re-read any note tied to a wrong answer.
- **Sit the mock exam in Section 4** under timed, closed-book conditions in Week 4. Treat your score as a readiness signal, not a guarantee.
- **Run the checklist in Section 5** in the final two days before your real exam.

1 The 4-Week Study Plan

About one focused hour per weekday. Weekends are reserved for a longer hands-on build block — that practical work is where most of the real learning happens.

The plan front-loads the two heaviest domains and saves the final week for consolidation and a full timed mock. Weekday sessions are deliberately short and single-topic so they fit into a busy schedule; weekend sessions are looser and project-driven. If you miss a weekday, push it to the weekend rather than skipping it.

Week	Theme	Domains covered	Weekday hrs	Weekend build
1	Foundations & Agentic Architecture	Primer + D1 (27%)	~5 hrs	Multi-step agent loop
2	Claude Code & Prompt Engineering	D2 (20%) + D3 (20%)	~5 hrs	Configure a real repo
3	Tools, MCP & Reliability	D4 (18%) + D5 (15%)	~5 hrs	Build a small MCP server
4	Consolidate, mock exam & revise	All five	~5 hrs	Full timed mock exam

Week 1 — Foundations & Agentic Architecture

Goal: get comfortable with how Claude works, then start the largest domain. Read the primer (2.1) before D1 (2.2).

Day	Focus (read in §2)	Task — ~1 hr	Checkpoint
Mon	Primer 2.1	What Claude is; messages API; system vs user; the request/response loop	Can describe a single API turn
Tue	Primer 2.1	Model families, context windows, token pricing, Batch API discount	Can pick a model on cost/latency
Wed	Domain 1	Tool-augmented single call vs. autonomous agent; when to use which	Know the decision rule
Thu	Domain 1	The agent loop: plan → act → observe → repeat; stopping conditions	Can sketch a loop on paper
Fri	Domain 1	Error recovery, fallbacks, graceful degradation, guardrails	List 3 failure modes + fixes
Sat	Build	Build a 3-step agent that calls 2 tools and recovers from one failure	Working agent + notes
Sun	Review	Redo the build's weak spot; attempt D1 practice questions (§3)	≥70% on D1 questions

Week 2 — Claude Code & Prompt Engineering

Goal: learn to configure Claude Code for real work, then design prompts that produce reliable, parseable output. Read 2.3 and 2.4.

Day	Focus (read in §2)	Task — ~1 hr	Checkpoint
Mon	Domain 2	Claude Code as an environment; CLAUDE.md purpose & best practices	Can write a CLAUDE.md
Tue	Domain 2	Hooks (event types), skills, and plugins — what each is for	Match hook to use case
Wed	Domain 2	Team setup: monorepos, permissions, compliance, MCP in Claude Code	Design a team config
Thu	Domain 3	System prompts, prefilling, XML-tagged inputs, role separation	Know each lever's effect
Fri	Domain 3	Structured output: JSON schema, tool_use for structure, anti-hallucination	Produce reliable JSON
Sat	Build	Add CLAUDE.md + one hook + one skill to a real (or sample) repo	Repo configured
Sun	Review	Prompt-caching basics; attempt D2 & D3 practice questions (§3)	≥70% on D2/D3

Week 3 — Tools, MCP & Reliability

Goal: design safe tools via MCP and master context/reliability techniques. Read 2.5 and 2.6.

Day	Focus (read in §2)	Task — ~1 hr	Checkpoint
Mon	Domain 4	What MCP is; client/server model; why it exists	Explain MCP in 3 sentences
Tue	Domain 4	Tool schema design: good vs. bad descriptions; scoping access	Critique a tool schema
Wed	Domain 4	MCP auth patterns; security pitfalls; least-privilege design	List 3 security controls
Thu	Domain 5	Context windows, token budgeting, what to keep vs. drop	Budget a long workflow
Fri	Domain 5	RAG architecture & prompt caching; coherence over long runs	Sketch a RAG pipeline
Sat	Build	Build a minimal MCP server exposing one well-scoped tool to Claude	Tool callable by Claude
Sun	Review	Attempt D4 & D5 practice questions (§3); list weak topics	≥70% on D4/D5

Week 4 — Consolidate, Mock Exam & Revise

Goal: turn knowledge into exam readiness. Lighter reading, heavier testing. Simulate real conditions.

Day	Focus (read in §2)	Task — ~1 hr	Checkpoint
Mon	All	Re-read notes for your two weakest domains only	Weak list shortened
Tue	All	Re-attempt every question you got wrong in §3	All previously-wrong fixed
Wed	All	Memorise the numbers: context sizes, Batch discount, pricing tiers	Recall from memory
Thu	All	Cross-domain scenarios: combine agent + MCP + context decisions	Reason across domains
Sat	Mock	Full mock exam (§4): 120-min timer, closed-book, no AI, no notes	Score recorded
Sun	Final	Review mock mistakes; run the revision checklist (§5)	Checklist complete

Milestones / go-no-go gates

- **End of Week 1:** you can sketch an agent loop and name its failure modes.
- **End of Week 2:** you can configure Claude Code and force structured JSON output.
- **End of Week 3:** you can describe an MCP server and a RAG pipeline from memory.
- **End of Week 4:** you score comfortably on the mock under real conditions. If not, delay the real exam a week and re-drill your weakest domain — the credential is only worth it if earned on your own knowledge.

2 Study Notes from First Principles

Everything below assumes no prior Claude experience. Concepts are introduced before they are used, with small concrete examples. Read each subsection on the day the plan schedules it.

2.1 Foundations Primer

What Claude is

Claude is a family of large language models made by Anthropic. At its core it does one thing: given a sequence of text (and sometimes images or documents), it predicts the most useful continuation. Everything else — chat, agents, tools, coding — is built on top of that single capability. Understanding this helps you reason about the exam: most “advanced” features are clever ways of *arranging the text you send* so the model behaves reliably.

The Messages API and conversation roles

You talk to Claude through the Messages API. A request is a list of messages, each with a **role**. The three roles you must know:

- **system** — standing instructions that frame the whole conversation (who Claude is, rules, format). Sent once, applies throughout.
- **user** — input from the person or your application (a question, a document, a tool result).
- **assistant** — Claude's replies. You can also *prefill* the start of an assistant turn to steer the output (more in 2.4).

A single “turn” is: you send the message list → Claude returns an assistant message. To hold a conversation, you resend the *entire* history each time, because the API itself is stateless — it has no memory between calls. Remembering this explains why context management (Domain 5) matters so much.

> **Worked example — the shape of a request**
>
> ```
> system: “You are a precise assistant. Answer in one sentence.”
> user: “What is the capital of France?”
> → assistant: “The capital of France is Paris.”
> ```
>
> To ask a follow-up, you send the system message, the first user message, the assistant reply, *and* the new user message — the whole list, every time.

Models, context windows, and tokens

Claude comes in several model tiers that trade off capability, speed, and cost. As an architect you choose per task: a smaller/faster model for high-volume simple work, a larger model for hard reasoning. Two numbers drive most design decisions:

- **Tokens** — the unit models read and bill in. A token is roughly 3–4 characters of English (about ¾ of a word). Both your input and Claude's output are counted.
- **Context window** — the maximum number of tokens (input + output) the model can consider at once. Exceed it and older content must be dropped or summarised. Modern Claude models have large windows (hundreds of thousands of tokens), but “large” is not “infinite,” and stuffing the window degrades both cost and focus.

Pricing intuition: you pay per million input tokens and per million output tokens, with output usually priced higher than input. Larger models cost more per token. Two cost levers the exam cares about: **prompt caching** (reuse a large, unchanging prefix cheaply across calls) and the **Batch API** (submit large non-urgent workloads asynchronously for a substantial discount). Know that these exist and when to reach for them; memorise the current rates from the official pricing page before exam day, since exact numbers change.

Exam tip

Cost-optimisation questions are common and usually scenario-based: “High volume, latency not critical, identical large instructions every call.” The expected answer combines a **smaller model + prompt caching + the Batch API**. Train yourself to spot the keywords (“high volume,” “not time-sensitive,” “repeated context”).

2.2 Domain 1 — Agentic Architecture & Orchestration (27%)

The biggest domain. It is fundamentally about *system design*: when to let Claude act autonomously, and how to keep that autonomy reliable.

Single call vs. agent — the core decision

A **tool-augmented single call** is: you ask once, Claude may call a tool or two, and returns an answer. Predictable and cheap. An **agent** is: Claude runs in a loop, deciding its own next step, calling tools, observing results, and continuing until a goal is met. Powerful, but less predictable and more expensive. The architect's rule of thumb:

- Use a **single call** when the task has a known, bounded shape (classify this, summarise that, answer from this document).
- Use an **agent** when the number and order of steps cannot be known in advance (research this topic, debug until tests pass, fulfil this open-ended request).
- Prefer the simplest design that works. Agents add failure surface; do not reach for one when a single structured call suffices.

The agent loop

Almost every agent is the same loop: **plan** → **act** → **observe** → **repeat**. Claude decides what to do (plan), calls a tool (act), receives the tool result back as a new message (observe), and decides again. The loop ends when a **stopping condition** is met.

Stopping conditions you must design deliberately

• **Goal reached** — Claude signals completion (e.g. produces a final answer).
• **Step / iteration cap** — a hard maximum number of loops to prevent runaway cost.
• **Budget cap** — stop when token or time spend exceeds a threshold.
• **No-progress detection** — stop or escalate if the agent repeats itself or gets stuck.
An agent with no stopping condition is a bug, not a feature.

Orchestration patterns

When one agent is not enough, you compose several. Know these patterns and when each fits:

Pattern	Shape	Use when
Single agent + tools	One loop, many tools	Most tasks; start here
Orchestrator–worker	A lead agent delegates subtasks to specialised workers	Tasks decompose into parallel or distinct sub-jobs
Prompt chaining	Fixed sequence of steps, each feeding the next	Steps are known and ordered (extract → transform → format)
Routing	A classifier sends the request to the right handler	Distinct request types need different handling

Pattern	Shape	Use when
Evaluator–optimiser	One agent produces, another critiques and loops	Quality matters and can be checked against criteria

Handoffs between agents are where reliability is won or lost. A clean handoff passes only the information the next agent needs — not the entire history — which keeps context small and focused (ties directly to Domain 5).

Reliability: error recovery and graceful degradation

Production agents fail constantly — a tool times out, returns garbage, or the model misreads a result. The exam rewards designs that expect this:

- **Retries with backoff** for transient tool/network failures, with a retry cap.
- **Fallbacks** — a secondary path when the primary fails (a cheaper model, a default answer, a human escalation).
- **Validation of tool output** before Claude acts on it, so bad data does not propagate.
- **Graceful degradation** — return a partial-but-useful result rather than crashing or looping forever.
- **Guardrails** — constrain what tools the agent can call and with what privileges (links to Domain 4 security).

> **Cost note that hides in agentic questions**
>
> Agentic workloads multiply token usage because every loop resends growing context. For **high-volume, non-urgent** agentic jobs, the **Batch API** and aggressive **prompt caching** of the stable instruction prefix are the standard cost controls. Watch for this in scenario stems.

2.3 Domain 2 — Claude Code Configuration & Workflows (20%)

Claude Code is Anthropic's agentic coding tool that operates inside your codebase from the terminal, IDE, or app. This domain tests configuring it for real teams, not toy demos.

CLAUDE.md — the project's standing instructions

CLAUDE.md is a file Claude Code reads automatically to learn how your project works. Think of it as the system prompt for your repository: conventions, commands, architecture notes, and things to avoid. Good practice keeps it concise and high-signal, because everything in it consumes context on every interaction.

- Put **stable, project-wide** facts here: build/test commands, code style, directory map, do-nots.
- Keep it **short and specific** — a bloated CLAUDE.md wastes context and dilutes attention.
- Files can be layered: a root CLAUDE.md plus per-directory files for local conventions.
- It is documentation Claude actually obeys, so treat changes with the same care as code.

Hooks, skills, and plugins — know the difference

Mechanism	What it is	Use it to
Hooks	Scripts that run automatically on lifecycle events (e.g. before/after a tool runs, on session start)	Enforce policy deterministically — run linters, block edits to protected paths, log actions
Skills	Packaged, reusable instructions + assets for a repeatable task	Teach Claude a repeatable workflow (e.g. “how we write migrations”)
Plugins	Distributable bundles that extend Claude Code	Share configuration, commands, or integrations across a team

The key distinction the exam probes

Hooks are deterministic; the model cannot skip them. If a requirement is non-negotiable (“never commit secrets,” “always run the formatter”), enforce it with a **hook**, not by asking nicely in a prompt or CLAUDE.md. Use instructions for guidance; use hooks for guarantees.

Configuring for a real team

Scenario questions love the phrase “a team of N engineers on a monorepo with compliance requirements.” The architecture answers usually combine:

- **Layered CLAUDE.md** files so shared conventions live at the root and local ones live per package.
- **Permission / allow-list configuration** so Claude Code can act freely on safe operations but is gated on risky ones.
- **Hooks for compliance** — protected-path guards, mandatory checks, audit logging.
- **MCP servers** wired in to give the whole team consistent access to internal tools/data (see 2.5).
- **Skills/plugins** distributed so every engineer gets the same workflows.

2.4 Domain 3 — Prompt Engineering & Structured Output (20%)

Not “write a nice prompt.” This domain is about prompt *architecture* that yields reliable, machine-parseable output at scale.

The core levers

- **System prompt** — set durable role, rules, and output contract here. It is the strongest, most persistent steering lever.

- **Clear, specific instructions** — state the task, constraints, and exact output format. Ambiguity is the main cause of unreliable output.
- **Examples (few-shot)** — show one or two worked input→output pairs to lock in format and style.
- **XML tags** — wrap inputs and sections in tags like <document>...</document> so Claude can tell instructions from data and you can target parts precisely.
- **Chain-of-thought steering** — ask Claude to reason step by step *before* the final answer for hard tasks; separate the reasoning from the answer so you can parse cleanly.
- **Prefilling** — start the assistant turn yourself (e.g. with `{`) to force a format or skip preamble.

Structured output — making results parseable

Applications need predictable shapes, not prose. Three reliable techniques, strongest last:

- **Ask for JSON and describe the schema** in the prompt, with an example. Simple, but the model can still drift.
- **Prefill the opening** of the JSON to force the format and suppress chatter.
- **Use tool_use / structured-output mode** — define the desired shape as a tool input schema; Claude must return arguments that conform. This is the most robust way to guarantee a parseable structure.

Anti-hallucination patterns the exam expects

- **Ground answers in provided context** and instruct Claude to say “I don't know” when the answer is not present.
- **Require citations / source spans** from the supplied material.
- **Constrain the output schema** so there is no room to invent fields.
- **Separate reasoning from the final structured answer** so validation is clean.

Prompt caching reappears here: a long, stable system prompt or instruction block (few-shot examples, schema, policy) can be cached so repeated calls reuse it cheaply. Place the stable, cacheable content first and the variable user content last.

2.5 Domain 4 — Tool Design & MCP Integration (18%)

How Claude reaches the outside world safely. The Model Context Protocol (MCP) is the centrepiece.

What a tool is

A **tool** is a function you describe to Claude — a name, a description, and an input schema. Claude does not run the tool; it *requests* a call with arguments, your code executes it, and you return the result as a new message. Claude then continues. The quality of your **tool descriptions and schemas** largely determines whether Claude uses tools correctly.

> **Good tool vs. bad tool**
>
> **Bad:** name `do_stuff`, description "handles data," a vague string parameter. Claude cannot know when or how to call it.
> **Good:** name `get_order_status`, description "Returns shipping status for one order; use only when the user gives an order ID," with a typed `order_id` parameter and clear constraints. Descriptions should tell Claude *when to use*, *when not to*, and *what each field means*.

What MCP is and why it exists

The **Model Context Protocol** is an open standard for connecting AI applications to external tools and data sources. Instead of hand-wiring every integration into every app, you run an **MCP server** that exposes tools (and resources/prompts) in a standard way, and any MCP-aware **client** (Claude Code, the Claude apps, your own app) can use them. The win is **standardisation and reuse**: build the integration once, use it everywhere.

- **Server** — exposes capabilities (tools, resources, prompts) over the protocol.
- **Client / host** — the AI application that connects to servers and lets Claude use them.
- **Transport** — how they communicate (e.g. local stdio, or a remote URL endpoint).

Security — the part the exam weights heavily

Giving a model access to external systems is a security decision. Expected best practices:

- **Least privilege** — expose the narrowest tool that does the job; never a broad "run any query" tool when a scoped one suffices.
- **Authentication & scoped credentials** — authenticate MCP connections and scope tokens to only what the server needs.
- **Validate and sanitise** tool inputs and outputs; treat tool results as untrusted data, since they can carry injected instructions.
- **Human-in-the-loop** for irreversible or high-impact actions (deletes, payments, production changes).
- **Audit logging** of tool calls for traceability.

> **Watch for prompt-injection via tool results**
>
> Content returned by a tool (a web page, a file, a database row) may contain text trying to hijack Claude. The architect-level answer is to **treat tool output as data, not instructions**, keep tools narrowly scoped, and gate destructive actions — not to trust that a clever prompt will prevent it.

2.6 Domain 5 — Context Management & Reliability (15%)

The smallest domain but, by reputation, the trickiest. It is about doing useful work within finite context, cheaply and coherently.

Token budgeting

Every call has a finite context window shared by input and output. As an architect you manage a **token budget**: decide what must be present (instructions, the current task, essential context) and what can be omitted, summarised, or fetched on demand. More context is not better — it costs more, slows responses, and can bury the signal that matters.

- **Keep**: the system prompt/contract, the immediate task, and the few facts needed now.
- **Summarise**: long histories or prior steps compressed to their essentials before continuing.
- **Retrieve on demand**: pull only the relevant chunks of a large corpus instead of pasting all of it.

RAG — retrieval-augmented generation

When the knowledge Claude needs is too large to fit (or changes often), use **RAG**: store documents in a searchable index, retrieve only the chunks relevant to the current query, and place those in context. This keeps the window small, the answers grounded, and the knowledge current without retraining. A minimal RAG pipeline:

> **RAG pipeline in five steps**
>
> **1.** Chunk documents sensibly. **2.** Index the chunks (e.g. embeddings in a vector store). **3.** On a query, retrieve the top relevant chunks. **4.** Insert them into the prompt as clearly-tagged context. **5.** Instruct Claude to answer *only* from that context and cite it.

Prompt caching & long-run coherence

Prompt caching stores a large, unchanging prefix so repeated requests reuse it at reduced cost and latency. Structure prompts so the stable part (instructions, schema, retrieved reference material that does not change between calls) comes first and is cacheable, and the variable user input comes last.

For **coherence across long workflows**, the reliable techniques are: periodic **summarisation** of what has happened, passing **only the needed state** across steps and handoffs, and using **structured intermediate artifacts** (e.g. a running plan or scratchpad) rather than relying on an ever-growing raw transcript. Smaller, well-curated context beats a giant one nearly every time.

3 Practice Questions by Domain

Twenty-five questions modelled on the style and difficulty of the five domains. These are original practice items written for this guide — not real exam questions — but they target the same judgement the exam rewards. Attempt them closed-book; the explained answer key follows in 3.6.

3.1 Domain 1 — Agentic Architecture & Orchestration

Q1. [Multiple choice] A workflow always follows the same three ordered steps: extract fields from a document, transform them, then format the result as JSON. Which design is most appropriate?

- **A.** A fully autonomous agent that decides each step at runtime
- **B.** Prompt chaining: three fixed steps, each feeding the next
- **C.** An orchestrator with five parallel worker agents
- **D.** A single call with no structure

Q2. [Scenario] An agent occasionally enters a loop where it repeats the same tool call without making progress, eventually exhausting your budget. What is the best architectural fix?

- **A.** Switch to a larger model
- **B.** Remove the tool so it cannot be called
- **C.** Add stopping conditions: an iteration cap and no-progress detection
- **D.** Increase the context window

Q3. [Multiple choice] You must decide between a tool-augmented single call and an autonomous agent. Which task most justifies an autonomous agent?

- **A.** Classifying support tickets into five fixed categories
- **B.** Summarising a single provided article
- **C.** Debugging a failing test suite until all tests pass
- **D.** Translating a paragraph to French

Q4. [Multiple choice] A lead agent splits a research request into independent sub-questions and assigns each to a specialised worker, then combines their findings. Which pattern is this?

- **A.** Routing
- **B.** Evaluator–optimiser
- **C.** Orchestrator–worker
- **D.** Prompt chaining

Q5. [Scenario] A high-volume nightly job runs an agentic pipeline over millions of records; latency is irrelevant but cost is critical, and each call shares a large, identical instruction prefix. Which combination best controls cost?

- **A.** Largest model + real-time calls
- **B.** Batch API + prompt caching of the stable prefix + a right-sized smaller model
- **C.** Disable tools to save tokens
- **D.** Increase the iteration cap

3.2 Domain 2 — Claude Code Configuration & Workflows

Q6. [Scenario] **Your team requires that secrets are never committed and the formatter always runs before a commit. How should you enforce this in Claude Code?**

- **A.** Write the rules in CLAUDE.md and trust Claude to follow them
- **B.** Add a few-shot example in the prompt
- **C.** Use hooks that run deterministically on the relevant events
- **D.** Ask the user to remember to check

Q7. [Multiple choice] **What is the primary purpose of a CLAUDE.md file?**

- **A.** To store API keys for the project
- **B.** To give Claude Code standing, project-specific instructions and conventions it reads automatically
- **C.** To replace the need for version control
- **D.** To log every action Claude takes

Q8. [Scenario] **A monorepo has shared org-wide conventions plus package-specific rules. What is the cleanest configuration?**

- **A.** One giant CLAUDE.md at the root with everything in it
- **B.** A root CLAUDE.md for shared conventions plus per-directory CLAUDE.md files for local rules
- **C.** No CLAUDE.md; rely on prompting each time
- **D.** A CLAUDE.md per engineer

Q9. [Multiple choice] **Which statement best distinguishes a skill from a hook in Claude Code?**

- **A.** Skills run deterministically on events; hooks are model-invoked workflows
- **B.** A skill packages a reusable workflow the model can apply; a hook runs deterministically on lifecycle events
- **C.** They are the same thing
- **D.** Hooks are for prompts; skills are for secrets

Q10. [Multiple choice] **You want every engineer on a team to share the same set of Claude Code commands and integrations. What is the most appropriate distribution mechanism?**

- **A.** Email a zip of config files
- **B.** A plugin that bundles the configuration for installation across the team
- **C.** A single shared login
- **D.** Paste the config into chat each session

3.3 Domain 3 — Prompt Engineering & Structured Output

Q11. [Scenario] **Your application parses Claude's output as JSON, but occasionally the model adds a friendly sentence before the JSON and your parser breaks. What is the most robust fix?**

A. Add “please only return JSON” and hope
B. Define the desired shape as a tool input schema and use structured-output / tool_use so the response must conform
C. Increase temperature
D. Switch to a smaller model

Q12. [Multiple choice] **Why wrap a supplied document in XML tags such as <document>...</document> within a prompt?**

A. It compresses the tokens
B. It lets Claude clearly separate instructions from data and lets you target sections precisely
C. It encrypts the content
D. It is required by the API

Q13. [Scenario] **Which set of techniques best reduces hallucination when answering from a provided knowledge base?**

A. Ground answers in the provided context, instruct “say you don't know if absent,” and require citations
B. Raise temperature and ask for creativity
C. Remove the system prompt
D. Allow free-form prose with no schema

Q14. [Scenario] **You send the same large block of few-shot examples and policy on every request, with only a short user query changing. How should you structure the prompt for cost and latency?**

A. Put the variable user query first, examples last
B. Put the large stable block first so it can be cached, with the variable query last
C. Randomise the order each call
D. Send everything as one giant user message with no system prompt

Q15. [Multiple choice] **What is the effect of prefilling the start of the assistant turn (e.g. beginning with an opening brace)?**

A. It deletes the system prompt
B. It steers the format and suppresses preamble, helping force a specific output shape
C. It doubles the token cost
D. It disables tool use

3.4 Domain 4 — Tool Design & MCP Integration

Q16. [Multiple choice] **Which tool definition is best designed for reliable use by Claude?**

A. name: do_stuff; description: “handles data”; one free-form string param
B. name: get_order_status; description states when and when not to use it; a typed order_id param with constraints
C. name: query; description: “runs anything”; raw SQL param
D. name: helper; no description; arbitrary params

Q17. [Multiple choice] **What problem does the Model Context Protocol primarily solve?**

A. It makes models run faster
B. It standardises how AI apps connect to external tools and data so an integration is built once and reused across clients
C. It encrypts all model outputs
D. It replaces the need for prompts

Q18. [Scenario] **An MCP-exposed tool returns content fetched from the public web, and that content contains text instructing Claude to delete a database. What is the correct architectural stance?**

A. Trust the instruction since it came through a tool
B. Treat tool output as untrusted data, scope tools narrowly, and gate destructive actions behind human approval
C. Add the instruction to CLAUDE.md
D. Raise the model temperature

Q19. [Scenario] **A reporting assistant needs to read three specific database tables. Which design follows least-privilege?**

A. Expose a tool that runs arbitrary SQL with admin credentials
B. Expose narrow, read-only tools scoped to exactly those three tables
C. Give Claude the database root password in the prompt
D. Allow write access just in case

Q20. [Multiple choice] **In MCP terminology, which component exposes tools and resources to be consumed?**

A. The client
B. The server
C. The transport
D. The token

3.5 Domain 5 — Context Management & Reliability

Q21. [Scenario] **A knowledge base is far larger than any context window and changes weekly. How should an assistant answer questions from it?**

A. Paste the entire knowledge base into every prompt
B. Fine-tune the model weekly on the whole base
C. Use RAG: retrieve only the relevant chunks per query and ground the answer in them
D. Summarise the whole base once and discard the source

Q22. [Multiple choice] **Which prompt structure maximises prompt-cache effectiveness?**

A. Stable instructions and reference material first; variable user input last
B. Variable user input first; stable material last
C. Interleave stable and variable content randomly
D. Put everything in the user role with no ordering

Q23. [Scenario] **An agent handing off to a specialised sub-agent should pass:**

A. The entire raw conversation transcript
B. Only the information the next agent needs for its subtask
C. Nothing; let it start blind
D. A copy of the system prompt only

Q24. [Scenario] **What is the most reliable way to keep a very long, multi-step workflow coherent?**

- **A.** Rely on an ever-growing raw transcript
- **B.** Periodically summarise progress and carry forward only needed state and structured artifacts
- **C.** Increase temperature each step
- **D.** Never summarise, to avoid losing information

Q25. [Multiple choice] **Which statement about context windows is correct?**

- **A.** A larger window means you should always fill it
- **B.** Input and output share the window, and overstuffing raises cost and can reduce focus
- **C.** The window only counts output tokens
- **D.** Windows are effectively infinite on modern models

3.6 Answer Key & Explanations

Mark yourself honestly. For every miss, re-read the matching note in Section 2 before moving on — understanding *why* matters more than the letter.

Q1 — Correct answer: B. The steps are **known and ordered**, so a fixed prompt chain is the simplest reliable design. Autonomy (A) adds failure surface for no benefit; parallel workers (C) are overkill for a strict sequence; an unstructured single call (D) sacrifices the reliability the fixed shape provides. Principle: prefer the simplest design that fits the task.

Q2 — Correct answer: C. Runaway loops are a missing-stopping-condition problem. An **iteration cap** plus **no-progress detection** (and ideally a budget cap) bound the behaviour. A bigger model (A) does not guarantee termination; removing the tool (B) breaks the task; a larger window (D) just lets it burn more before failing.

Q3 — Correct answer: C. Debugging-until-green has an **unknown number and order of steps** — the model must investigate, try fixes, and re-check iteratively. That open-endedness is exactly what agents are for. The others (A, B, D) are bounded, single-shape tasks best served by a single structured call.

Q4 — Correct answer: C. A lead delegating distinct sub-jobs to specialised workers and aggregating results is the **orchestrator–worker** pattern. Routing (A) merely directs a request to one handler; evaluator–optimiser (B) is produce-then-critique; chaining (D) is a fixed linear sequence.

Q5 — Correct answer: B. “High volume, latency irrelevant, repeated large prefix” is the canonical signal for the **Batch API** (async discount) plus **prompt caching** of the unchanging prefix, on the **smallest model** that meets quality. (A) maximises cost; (C) breaks functionality; (D) raises cost.

Q6 — Correct answer: C. Non-negotiable guarantees belong in **hooks**, which run deterministically and cannot be skipped by the model. CLAUDE.md and prompt examples (A, B) are guidance, not guarantees; manual checks (D) are unreliable. Rule: instructions for guidance, hooks for guarantees.

Q7 — Correct answer: B. CLAUDE.md is the project's **standing instructions** — conventions, commands, architecture notes — read automatically each session. It is not a secrets store (A), not a VCS replacement (C), and not an audit log (D, which a hook would handle).

Q8 — Correct answer: B. **Layered** CLAUDE.md files keep shared rules at the root and local rules near the code they govern, staying concise and high-signal. One giant file (A) bloats context; none (C) loses consistency; per-engineer files (D) fragment shared standards.

Q9 — Correct answer: B. A **skill** packages reusable instructions/assets for a repeatable task the model applies; a **hook** fires deterministically on lifecycle events. (A) reverses them; (C) and (D) are wrong.

Q10 — Correct answer: B. **Plugins** are the distributable bundles for sharing Claude Code configuration/commands across a team. Manual zips (A) and pasting (D) do not scale or stay consistent; a shared login (C) is a security problem.

Q11 — Correct answer: B. The most reliable guarantee of a parseable shape is to express it as a **tool input schema** and use **structured-output / tool_use**, so the model must return conforming arguments. Politely asking (A) still drifts; temperature (C) is unrelated and higher hurts; model size (D) does not fix format. Prefilling the opening brace is a good lighter-weight alternative to A.

Q12 — Correct answer: B. XML tags create an unambiguous boundary between **instructions and data** and let you reference sections precisely. They do not compress (A), encrypt (C), or constitute an API requirement (D).

Q13 — Correct answer: A. Grounding in supplied context, an explicit **“I don't know”** instruction, and **required citations** are the standard anti-hallucination patterns. (B), (C), and (D) all increase the room to invent.

Q14 — Correct answer: B. Place the **stable, cacheable** content (examples, policy, schema) **first** and the variable user input **last**, so **prompt caching** can reuse the prefix cheaply across calls. Order matters for cache hits, which rules out (A) and (C).

Q15 — Correct answer: B. Prefilling **steers the output format** and skips chatty preamble — useful for forcing JSON or a particular structure. It does not affect the system prompt (A), double cost (C), or disable tools (D).

Q16 — Correct answer: B. A good tool has a **specific name**, a description that says **when and when not to use it**, and a **typed, constrained schema**. (A) and (D) are too vague to invoke correctly; (C) is both vague and a security hazard (arbitrary SQL).

Q17 — Correct answer: B. MCP is an **open standard** for connecting AI applications to tools/data, enabling **build-once, reuse-everywhere** integrations. It is not a performance (A), encryption (C), or prompt-replacement (D) technology.

Q18 — Correct answer: B. This is **prompt injection via tool results**. The architect treats tool output as **data, not instructions**, keeps tools **least-privilege**, and puts **human approval** on irreversible actions. (A) is the vulnerability itself; (C) and (D) make things worse.

Q19 — Correct answer: B. **Least privilege** means the narrowest capability that does the job: **read-only, table-scoped** tools. Arbitrary SQL (A), root credentials in-prompt (C), and unnecessary write access (D) all violate it and expand the blast radius.

Q20 — Correct answer: B. The **server** exposes capabilities (tools, resources, prompts). The **client/host** consumes them; the **transport** is the communication channel. “Token” (D) is unrelated here.

Q21 — Correct answer: C. **RAG** fits oversized, changing corpora: index, **retrieve relevant chunks per query**, and ground the answer. Pasting everything (A) busts the window and cost; weekly fine-tuning (B) is slow and expensive; a one-time summary (D) loses detail and currency.

Q22 — Correct answer: A. Caching reuses an unchanging **prefix**, so the **stable content goes first** and the variable input last. The other orderings break or never form a reusable prefix.

Q23 — Correct answer: B. Clean handoffs pass **only what the next agent needs**, keeping context small and focused. The full transcript (A) wastes context and dilutes attention; nothing (C) loses necessary state; system-prompt-only (D) omits the task specifics.

Q24 — Correct answer: B. Long-run coherence comes from **periodic summarisation**, carrying **only needed state**, and using **structured intermediate artifacts** (a running plan/scratchpad). A growing raw transcript (A, D) bloats context and buries signal; temperature (C) is irrelevant.

Q25 — Correct answer: B. The window is shared by **input and output**, and **overstuffing** raises cost/latency and can dilute the model's attention. Bigger is not a mandate to fill (A); output-only (C) is wrong; no window is infinite (D).

4 Full-Length Mock Exam

Forty questions weighted to mirror the five domains. Sit this in Week 4 under real conditions: a 120-minute timer, closed-book, no Claude, no notes. Original practice items — not real exam questions — built to rehearse the same judgement.

Exam conditions — set these before you start

- Timer: **120 minutes** (the real exam is 60 questions in 120 min; this 40-question mock scales to roughly 80 minutes — give yourself 80 and treat finishing early as a good sign).
- **Closed-book.** No documentation, no Claude, no web. Phone away.
- Write your letter answers on paper, then grade against 4.2. Target: comfortably clear, with no single domain dragging.

Q1. [Multiple choice] **A customer-support flow must, in a fixed order, (1) detect language, (2) classify intent, (3) draft a reply. Best design?**

- **A.** Autonomous agent
- **B.** Prompt chaining
- **C.** Five parallel workers
- **D.** Single unstructured call

Q2. [Multiple choice] **Which is the clearest signal to choose an autonomous agent over a single call?**

- **A.** Output must be JSON
- **B.** The number and order of steps is unknown in advance
- **C.** High request volume
- **D.** Low latency requirement

Q3. [Scenario] **An agent keeps running after the task is done, wasting tokens. What was missing?**

- **A.** A larger model
- **B.** A goal/completion stopping condition
- **C.** More tools
- **D.** A bigger context window

Q4. [Multiple choice] **Lead agent delegates distinct subtasks to specialists and merges results. Pattern?**

- **A.** Routing
- **B.** Orchestrator–worker
- **C.** Evaluator–optimiser
- **D.** Chaining

Q5. [Multiple choice] **One agent drafts, a second critiques and sends it back to improve. Pattern?**

- **A.** Routing
- **B.** Orchestrator–worker
- **C.** Evaluator–optimiser
- **D.** Single call

Q6. [Scenario] **Nightly, non-urgent, millions of records, identical large prefix. Cheapest sound design?**

- **A.** Largest model, real-time
- **B.** Batch API + prompt caching + right-sized model
- **C.** Remove all tools
- **D.** Raise iteration cap

Q7. [Scenario] **A tool call sometimes times out transiently. Best first-line handling?**

- **A.** Crash the whole agent
- **B.** Retry with backoff up to a cap, then fall back
- **C.** Ignore and continue with no data
- **D.** Switch models

Q8. [Multiple choice] **Why validate a tool's output before the agent acts on it?**

- **A.** To save tokens
- **B.** To stop bad/garbage data from propagating through the loop
- **C.** It is required by the API
- **D.** To increase temperature

Q9. [Multiple choice] **“Graceful degradation” in an agent means:**

- **A.** Crash cleanly
- **B.** Return a partial-but-useful result instead of failing entirely
- **C.** Retry forever
- **D.** Escalate every error to a human

Q10. [Scenario] **Cleanest way to keep multi-agent context small?**

- **A.** Pass full transcript everywhere
- **B.** Hand off only the data the next agent needs
- **C.** Duplicate the system prompt
- **D.** Never summarise

Q11. [Multiple choice] **Which task is LEAST suited to an autonomous agent?**

- **A.** Open-ended research
- **B.** Debug until tests pass
- **C.** Classify a ticket into 5 fixed labels
- **D.** Multi-step task fulfilment

Q12. [Scenario] **A rule must never be skipped (e.g. block edits to /secrets). Enforce via:**

- **A.** CLAUDE.md note
- **B.** A hook on the relevant event
- **C.** A few-shot example
- **D.** Verbal reminder

Q13. [Multiple choice] **Primary role of CLAUDE.md:**

- **A.** Secret storage
- **B.** Standing project instructions Claude reads automatically
- **C.** Audit log
- **D.** Replace git

Q14. [Scenario] **Monorepo with org-wide + package-specific conventions. Best layout?**

- **A.** One huge root file
- **B.** Root CLAUDE.md + per-directory CLAUDE.md files
- **C.** No config
- **D.** Per-engineer files

Q15. [Multiple choice] **A skill in Claude Code is:**

- **A.** A deterministic event script
- **B.** A packaged reusable workflow the model can apply
- **C.** A secret vault
- **D.** A billing setting

Q16. [Multiple choice] **Best way to share identical Claude Code commands across a team:**

- **A.** Email zips
- **B.** A plugin bundle
- **C.** Shared login
- **D.** Paste each session

Q17. [Multiple choice] **Why keep CLAUDE.md concise?**

- **A.** Disk space
- **B.** It consumes context every interaction; bloat dilutes attention and raises cost
- **C.** API limit on file size
- **D.** It is encrypted

Q18. [Multiple choice] **Hooks differ from instructions because hooks are:**

- **A.** Suggestions
- **B.** Deterministic and cannot be skipped by the model
- **C.** Slower
- **D.** Model-generated

Q19. [Scenario] **To give a whole team consistent access to an internal data source inside Claude Code:**

- **A.** Hardcode it in each prompt
- **B.** Wire in an MCP server
- **C.** Share a password
- **D.** Disable tools

Q20. [Scenario] **Output must be parseable JSON every time. Most robust technique?**

- **A.** Ask politely for JSON
- **B.** Define a tool input schema and use structured output / tool_use
- **C.** Raise temperature
- **D.** Smaller model

Q21. [Multiple choice] **Purpose of wrapping inputs in XML tags:**

- **A.** Compression
- **B.** Separate instructions from data; target sections precisely
- **C.** Encryption
- **D.** API requirement

Q22. [Scenario] **Strongest anti-hallucination combo for KB answering:**

- **A.** Ground in context + “say I don't know” + require citations
- **B.** Higher temperature
- **C.** Remove system prompt
- **D.** Free-form prose

Q23. [Multiple choice] **To force JSON and suppress preamble cheaply, you can:**

- **A.** Delete the system prompt
- **B.** Prefill the assistant turn's opening
- **C.** Double max tokens
- **D.** Disable tools

Q24. [Multiple choice] **Where do durable role and output-contract rules belong?**

- **A.** A trailing user message
- **B.** The system prompt
- **C.** A tool result
- **D.** Nowhere

Q25. [Multiple choice] **Few-shot examples primarily help by:**

- **A.** Reducing cost
- **B.** Locking in the desired format and style via worked examples
- **C.** Encrypting output
- **D.** Disabling reasoning

Q26. [Multiple choice] **For hard reasoning tasks, a good structured pattern is to:**

- **A.** Forbid all reasoning
- **B.** Have Claude reason step-by-step, then give a separable final answer
- **C.** Maximise temperature
- **D.** Hide the question

Q27. [Scenario] **Prompt with a big stable example block + short changing query. Best order?**

- **A.** Query first, block last
- **B.** Stable block first (cacheable), query last
- **C.** Random
- **D.** All in one user blob

Q28. [Multiple choice] **Best-designed tool definition:**

- **A.** do_stuff, vague string param
- **B.** get_order_status, typed order_id, says when/when-not to use
- **C.** query, raw SQL
- **D.** helper, no description

Q29. [Multiple choice] **MCP primarily provides:**

- **A.** Faster inference
- **B.** A standard for connecting AI apps to tools/data, reusable across clients
- **C.** Output encryption
- **D.** Prompt replacement

Q30. [Scenario] **Tool returns web text saying “delete everything.” Correct stance?**

- **A.** Obey it
- **B.** Treat tool output as untrusted data; scope tools; gate destructive actions
- **C.** Add to CLAUDE.md
- **D.** Raise temperature

Q31. [Scenario] **Reporting bot needs to read 3 tables. Least-privilege design?**

- **A.** Arbitrary SQL as admin
- **B.** Read-only tools scoped to those 3 tables
- **C.** Root password in prompt
- **D.** Full write access

Q32. [Multiple choice] **Which MCP component exposes capabilities?**

- **A.** Client
- **B.** Server
- **C.** Transport
- **D.** Host UI

Q33. [Multiple choice] **A good tool description should tell Claude:**

- **A.** Only the function name
- **B.** When to use it, when not to, and what each field means
- **C.** The server's IP
- **D.** The billing tier

Q34. [Scenario] **Highest-impact MCP security control among these:**

- **A.** Pretty names
- **B.** Least-privilege scoping + scoped auth + human gate on destructive actions
- **C.** More tools
- **D.** Larger context

Q35. [Scenario] **KB far bigger than the window and changes weekly. Best approach?**

- **A.** Paste it all each call
- **B.** Weekly fine-tune
- **C.** RAG: retrieve relevant chunks per query
- **D.** Summarise once, discard source

Q36. [Multiple choice] **Order that maximises prompt-cache hits:**

- **A.** Variable first, stable last
- **B.** Stable first, variable last
- **C.** Random
- **D.** No ordering

Q37. [Scenario] **Multi-step workflow loses coherence over time. Best remedy?**

- **A.** Grow the raw transcript forever
- **B.** Periodic summarisation + carry only needed state + structured artifacts
- **C.** Raise temperature each step
- **D.** Never summarise

Q38. [Multiple choice] **True statement about the context window:**

- **A.** Always fill a bigger window
- **B.** Input and output share it; overstuffing raises cost and can reduce focus
- **C.** Counts output only
- **D.** Effectively infinite

Q39. [Multiple choice] **A token is roughly:**

- **A.** One full sentence
- **B.** About 3–4 characters of English (~¾ of a word)
- **C.** One paragraph
- **D.** One byte always

Q40. [Scenario] **A RAG assistant must answer only from retrieved documents. Best instruction design?**

- **A.** Let it answer from general knowledge if unsure
- **B.** Instruct it to answer only from the provided context, cite sources, and abstain if the answer is absent
- **C.** Remove the retrieved context to save tokens
- **D.** Raise temperature for creativity

4.2 Mock Exam — Answer Key

Grade yourself, then bucket your misses by domain tag to see exactly where to revise.

#	Dom	Ans	Why
1	D1	B	Fixed ordered steps → prompt chaining.
2	D1	B	Unknown step shape is the defining agent signal.
3	D1	B	Needs an explicit completion stopping condition.
4	D1	B	Delegate-and-merge = orchestrator–worker.
5	D1	C	Produce-then-critique loop = evaluator–optimiser.
6	D1	B	Batch + caching + smaller model for bulk non-urgent work.
7	D1	B	Bounded retries with backoff, then fallback.
8	D1	B	Validation prevents bad data propagating.
9	D1	B	Partial useful output beats total failure.
10	D1	B	Minimal, targeted handoffs.
11	D1	C	Bounded classification needs no autonomy.
12	D2	B	Guarantees require deterministic hooks.
13	D2	B	Project standing instructions.
14	D2	B	Layered CLAUDE.md files.
15	D2	B	Reusable packaged workflow.
16	D2	B	Plugins distribute team config.
17	D2	B	It costs context every turn.
18	D2	B	Deterministic, unskippable.
19	D2	B	MCP server for shared tool/data access.
20	D3	B	Schema-enforced structured output.
21	D3	B	Clear instruction/data separation.
22	D3	A	Grounding + abstain + citations.
23	D3	B	Prefilling steers format.
24	D3	B	System prompt is the durable steering lever.
25	D3	B	Examples anchor format/style.
26	D3	B	Reason first, separable final answer.
27	D3	B	Stable prefix first for caching.
28	D4	B	Specific, typed, well-described, scoped.
29	D4	B	Standard, reusable integrations.
30	D4	B	Tool output is data; gate destructive ops.
31	D4	B	Narrow read-only scoped tools.
32	D4	B	The server exposes tools/resources.
33	D4	B	Usage guidance + field semantics.
34	D4	B	Least privilege + auth + human gate.
35	D5	C	RAG fits large, changing corpora.
36	D5	B	Cache reuses a stable prefix.
37	D5	B	Summarise, minimal state, artifacts.
38	D5	B	Shared budget; overstuffing hurts.
39	D5	B	~3–4 chars / ¾ word.
40	D5	B	Ground strictly in retrieved context; cite and abstain.

Reading your result

Count correct answers per domain tag (D1–D5). Any domain where you miss a third or more is your priority for the final revision days. Because D1 and D2 together are ~47% of the real exam, weakness there hurts most. Re-take this mock after revising and confirm the weak domain has recovered before booking the real exam.

5 Final Revision Checklist

Run this in the last two days. If you cannot tick a box from memory, that is your revision target. Boxes are grouped by domain and by exam-day logistics.

Domain 1 — Agentic Architecture (27%)

- I can state the rule for single call vs. autonomous agent.
- I can draw the plan→act→observe loop and name 3+ stopping conditions.
- I can name the orchestration patterns and when each fits.
- I can list failure modes (timeout, bad output, loops) and their fixes (retry/backoff, fallback, validation, degradation).
- I know Batch API + caching apply to high-volume non-urgent agentic jobs.

Domain 2 — Claude Code (20%)

- I can explain what CLAUDE.md is and write a concise one.
- I can distinguish hooks vs. skills vs. plugins and pick the right one.
- I know to enforce guarantees with hooks, not instructions.
- I can design a layered config for a team monorepo with compliance.

Domain 3 — Prompt Engineering & Structured Output (20%)

- I can name the levers: system prompt, examples, XML tags, CoT, prefilling.
- I know structured output / tool_use is the most robust way to force a parseable shape.
- I can list the anti-hallucination patterns (grounding, abstain, citations, schema).
- I know stable content goes first for prompt caching.

Domain 4 — Tool Design & MCP (18%)

- I can describe a good vs. bad tool definition.
- I can explain MCP (server / client / transport) and why it exists.
- I can state least-privilege design and the prompt-injection-via-tool-output risk.
- I know destructive actions need a human gate.

Domain 5 — Context & Reliability (15%)

- I can explain token budgeting and that input + output share the window.
- I can sketch a 5-step RAG pipeline from memory.
- I can explain prompt caching ordering and long-run coherence techniques.

Numbers to recall cold (verify on the official pages first)

- Approximate token-to-word ratio (~3–4 chars per token).
- Current model tiers and their relative cost/speed trade-offs.
- That output tokens usually cost more than input tokens.
- That the Batch API gives a substantial discount for async bulk work.
- Context window order of magnitude for the models you would choose.

Exam-day logistics

- Confirmed exam date, time, and that my system meets proctoring requirements.
- Quiet, well-lit room; ID ready; no second monitor or phone.
- Plan: ~2 min/question; flag-and-return on hard ones; never leave blanks.
- Closed-book mindset rehearsed via the mock under real conditions.
- Slept well — fatigue costs more points than one more re-read.

6 Resources & Next Steps

Primary sources first. The official materials are the source of truth; everything in this guide should be checked against them.

Official & primary

- **Anthropic Academy (Skilljar)** — the free courses that map to the exam domains (start with “Building with the Claude API,” then the MCP and Claude Code courses). This is also where you register for the exam.
- **The official CCA-Foundations Exam Guide** — download it from the Academy portal and treat its objectives and weightings as authoritative over this document.
- **Claude API documentation** — tool use, structured output, prompt caching, the Batch API, pricing.
- **Claude Code documentation** — CLAUDE.md, hooks, skills, plugins, MCP setup.
- **Model Context Protocol specification** — server/client architecture and tool schemas.
- **Claude Partner Network** — joining is free and may waive the exam fee for early members.

How to verify a fact in this guide

Anything quantitative (pricing, context-window sizes, the Batch discount, the pass mark, exact domain weightings) can change. Before relying on a number for study or on exam day, confirm it on the official Anthropic pricing, docs, and Exam Guide. Concepts in this guide are stable; specific numbers are the thing to re-check.

The most important preparation of all

This exam rewards **hands-on experience** over reading. The three weekend builds in the plan — an agent loop, a configured repo, and a small MCP server — are not optional extras; they are where the concepts become intuition. If you do only one thing beyond reading, build those three. An architect is defined by having designed real systems, and the proctored, no-assistance format exists precisely to reward that.

A note on integrity

The exam is closed-book and prohibits AI assistance for a reason: the credential is only worth something if it reflects what *you* can do. Use this guide to genuinely learn the material, sit the exam on your own knowledge, and the certification will mean what it is supposed to mean to the people who see it.

End of study package • Built May 2026 • Unofficial — not affiliated with, endorsed by, or sponsored by Anthropic. Verify all specifics against official Anthropic sources before exam day.